YOLOv8-Based Object Detection with Kalman Filter Optimization for Ball Tracking in Wheeled Soccer Robot

Fikri Rivandi[1*], Feri Candra[2]
[1,2]Dept. of Informatics Engineering, Faculty of Engineering, Universitas Riau, Pekanbaru, Indonesia
[1]fikri.rivandi2583@student.unri.ac.id, [2]feri@eng.unri.ac.id

*Corresponding author: fikri.rivandi2583@student.unri.ac.id

Abstract—Deep learning-based object detection, such as YOLO, significantly improves accuracy in KRSBI-Beroda robots but introduces heavy computational loads that lead to high latency and movement instability on limited hardware. This research proposes a tracking optimization system that integrates YOLOv8 with a Kalman Filter using a frame-skipping strategy. In this architecture, YOLOv8 serves as a periodic detector while the Kalman Filter acts as a recursive estimator to maintain tracking continuity during skip phases. Experimental results on an ASUS K401U laptop demonstrate a substantial increase in processing speed from 13.60 FPS to 38.77 FPS. Furthermore, the system achieved zero tracking loss during occlusion scenarios and reduced mission completion time by 62%. This integration provides a robust solution for real-time robotic vision in dynamic environments.

Keywords: KRSBI-Beroda, YOLOv8, Kalman Filter, Frame Skipping, Object Tracking

---

INTRODUCTION

The Engineering Robotic Club (ERC) of Universitas Riau (UNRI) has consistently participated in the Wheeled Indonesian Soccer Robot Contest (KRSBI Beroda). A critical challenge in this competition is the robot's ability to track the ball and goals autonomously under dynamic conditions. While deep learning methods like YOLO offer superior accuracy compared to traditional HSV filtering, they demand high computational resources.

On the current hardware platform ASUS K401U with an Intel Core i5 7200U and Nvidia 940MX, continuous YOLO inference creates a computational bottleneck, leading to reduced frame rates and delayed actuator responses. In addition, frequent occlusion conditions, where the ball is partially or completely obstructed, often result in immediate tracking failure. To address these issues, this study integrates a Kalman Filter to estimate the trajectory of the ball, enabling the robot to maintain positional awareness even during detection gaps or occlusion events, thereby producing smoother and more efficient movement.

METHODOLOGY

A. YOLOv8 Implementation

The system utilizes a YOLOv8 model trained on 1,125 images captured via the robot's omnidirectional camera. The detector outputs a bounding box in the format (x_c, y_c, w, h, C), which is then passed as the Measurement Vector (Z_k) to the Kalman Filter.

B. Kalman Filter Configuration

The state vector (X) is defined to encompass position, velocity, and object dimensions:

$$X = [x, y, v_x, v_y, w, h]^T$$

The transition matrix (F) is modeled based on constant velocity assumptions:

$$F = \begin{bmatrix} 1 & 0 & dt & 0 & 0 & 0 \\ 0 & 1 & 0 & dt & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The observation matrix (H) maps the state space to the measurement space, focusing on position and dimensions since velocity is not directly measured.

C. Prediction and Frame Skipping Strategy

To optimize hardware usage, the system implements a two-tiered approach:

1. Prediction Phase: The Kalman Filter projects the next state based on the previous motion model.
2. Correction Phase: The YOLOv8 detector is invoked every several frames (S_L) to provide new measurements for the update step. During frames where $frame_count \bmod S_L \neq 0$, the system relies solely on the Kalman Filter prediction, thereby significantly reducing the frequency of neural network inference.

RESULT AND DISCUSSION

A. Data Collecting

The dataset collection process is conducted to initiate model development. As a result, a total of 1,125 images consisting of ball and goal images were obtained. The images were captured directly using an omnidirectional camera. The ball and goalpost were placed in different positions in each image to ensure a diverse dataset. Some examples of the collected dataset can be seen in Figures 5 and 6.

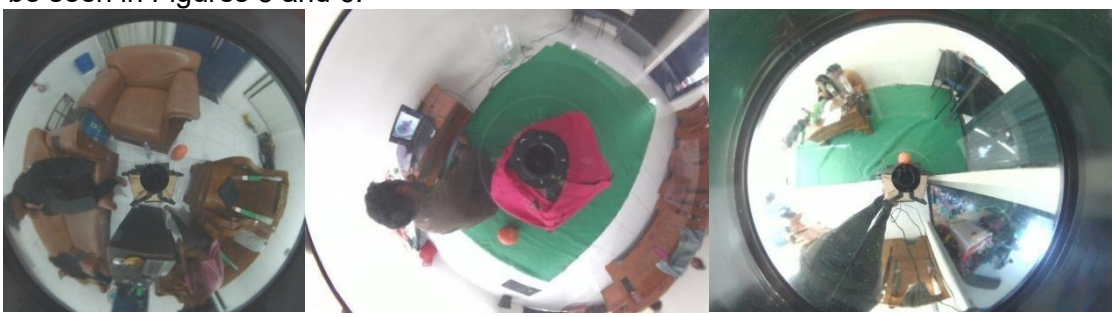


Figure 5. Some of ball datasets

Figure 6. Some of goalpost datasets

All collected datasets were labeled. Labeling is the process of assigning labels to a set of pixels that form an object within an image. Different objects are given different labels to distinguish them. This process is crucial for shape analysis and pattern recognition in image processing, as it helps the model accurately identify and differentiate objects within a dataset [9]. This labeling process involves marking the ball and goal objects with bounding boxes. The coordinates of these boxes are then extracted and saved in a .txt file format. An example of a labeled dataset can be seen in Figure 7.

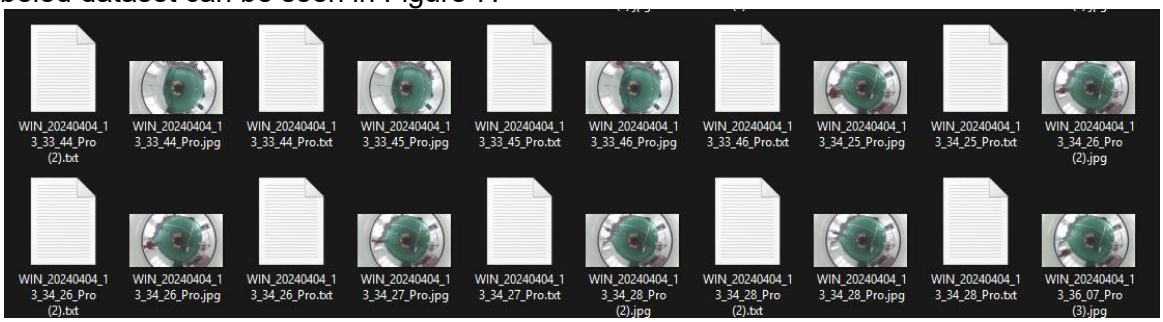


Figure 7. Some of labeled datasets

B. Data Preprocessing and Augmentation

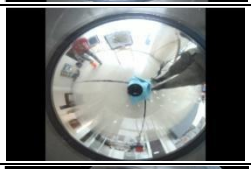
After the labeling process, the dataset undergoes to preprocessing. Preprocessing is the stage of processing collected image data to make necessary adjustments [10]. The First step for preprocessing is resizing images to 600x400 resolution. Resizing images is essential to ensure that all images have consistent dimensions before being fed into the model, as YOLO requires fixed-size input images. This process helps improve processing efficiency and maintains consistency within the model.

Figure 8. Resizing image from 3840x2160 (left image) to 600x400 (right image)

Following this, image augmentation is applied. Image augmentation is an effective way to improve image classification models [11] [12]. This process artificially expand the dataset, enhancing the learning process and overall performance by inputting diverse data variations into the model [13]. The augmentation techniques applied include horizontal flipping, which mirrors the image to introduce variations in object patterns. Rotation is also used, with images being rotated 90 degrees clockwise, 90 degrees counterclockwise, 45 degrees clockwise, and 45 degrees counterclockwise. This allows the model to learn to recognize objects in different orientations. Additionally, brightness adjustment is applied by increasing the brightness by 20% and decreasing it by 20%, ensuring the model is more resilient to realworld lighting variations. All these augmentation techniques aim to make the model more robust in detecting objects under various conditions encountered during testing. The results of this augmentation are illustrated in the following table.

Table 1. Illustration of Augmentation Result

Augmentation Process	Original Image	After Augmentation

Horizontal Flip		
90º clockwise rotation		
90º counterclockwise rotation		
45º clockwise rotation		
45º counterclockwise rotation		
Brightness +20%		
Brightness -20%		

After preprocessing and augmentation, the dataset is divided into two groups: training data and validation data. The data is split with an 80% allocation for training and 20% for validation. This division is essential to ensure that the model learns effectively while also being evaluated on unseen data. The training set is used to adjust the model's parameters, helping it recognize patterns and features in the images. Meanwhile, the validation set is used to assess the model's performance and detect potential overfitting, where the model performs well on training data but struggles with new, unseen data. By maintaining this 80:20 ratio, the model can generalize better, improving its ability to detect balls and goals under various real-world conditions.

C. Model Training

The object detection model training process using the YOLOv8 algorithm is conducted with the Ultralytics library, which provides an efficient and user-friendly interface for training YOLO (You Only Look Once) models. This library is specifically designed to support the latest versions of YOLO, such as YOLOv8, and facilitates real-time implementation. This real-time capability is crucial for systems that require fast and accurate object detection, such as robotic vision applications. YOLOv8 includes built-in data augmentation techniques to improve model generalization. Mosaic augmentation combines four images into one for better object diversity, MixUp blends images to enhance robustness, Random Perspective adjusts object positions to simulate different viewpoints, and Color Jittering modifies brightness, contrast, and saturation to improve performance under varying lighting conditions. These augmentations help the model detect objects more accurately across different environments [14].

For learning rate, YOLOv8 employs an adaptive learning strategy, starting at 0.001 and dynamically adjusting using a cosine annealing scheduler. This prevents overshooting or slow convergence by gradually reducing the learning rate as training progresses, ensuring stable optimization without manual tuning [14]. The YOLOv8 model training is conducted using the Google Colab platform. This platform is chosen because it provides access to a T4 GPU, which offers excellent performance for model training, ensuring that the process does not take too long. The dataset is trained for 100 epochs with a batch size of 16, optimizing the model's ability to learn object features effectively.

When training the YOLOv8 model using the Ultralytics library, the command used is as follows:

!yolo task=detect mode=train model=yolov8n.pt data=(data_path) epochs=100 batch=16

This command is used to train the object detection model with several important parameters. The task=detect parameter specifies that the task being performed is object detection. The mode used is train, indicating that the model will be trained using the prepared dataset. The base model selected is yolov8n.pt, a lightweight version of YOLOv8, which is suitable for resource-limited environments or applications that require high-speed inference. The data=(data_path) parameter defines the location of the dataset configuration file. This file contains essential information such as the number of object classes (ball and goal), the location of the images, and their corresponding labels. Proper configuration of this dataset ensures that the model correctly learns to detect the specified objects. The epochs=100 parameter determines the number of training iterations, which influences how well the model learns from the dataset. A higher number of epochs allows the model to refine its predictions through repeated training cycles. Additionally, the batch=16 parameter defines how many images are processed in one batch during training. A larger batch size can speed up the training process but requires more memory, making it essential to balance computational efficiency and performance.

D. Model Evaluation

After completing the model training process, a confusion matrix is obtained from the bestperforming epoch during training. Confusion matrix is a commonly used accuracy measurement method in data mining. It is presented as a table that displays the number of correctly and incorrectly classified data points after the classification process[15].

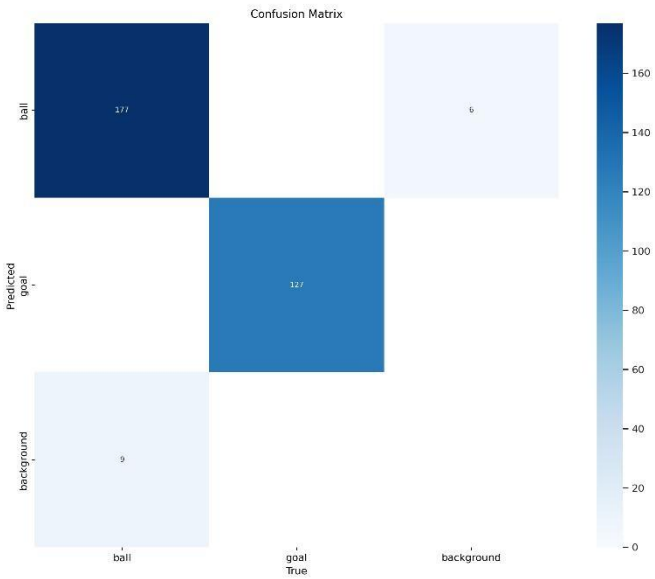


Figure 9. Confusion Matrix

Based on Figure 9, the results indicate that the trained YOLOv8 model performs well in classifying objects. Images containing a ball are correctly detected as a ball, and images containing a goal are accurately predicted as a goal, with very high values. This signifies that the model can classify objects with a high degree of accuracy. Additionally, the number of misclassified images is very small compared to the total predicted images, demonstrating that classification errors have been effectively minimized. By definition, Accuracy is the ratio of correctly classified data to the total dataset [16]. To determine the accuracy, we first define the values in the confusion matrix:

- True Positives (TP) : 177 (correctly classified as ball) + 127 (correctly classified as goalpost)
- True Negatives (TN) : 0 (correctly classified as background)
- False Positives (FP) : 6 (misclassified background as ball) False Negatives (FN) : 9 (misclassified ball as background)

The exact accuracy value is calculated using the following formula:

$$\text{Accuracy} = \frac{\text{폳} + \text{폳}}{\text{폳} + \text{폳} + \text{퐹} + \text{퐹}} \times 100\%$$

$$= \frac{175+127+0}{175+127+0+6+9} \times 100\%$$

$$= \frac{302}{317} \times 100\%$$

$$= 95{,}87\%$$

Apart from the confusion matrix, the model training also produces several other curves, such as the precision curve, recall curve, and F1-score curve.

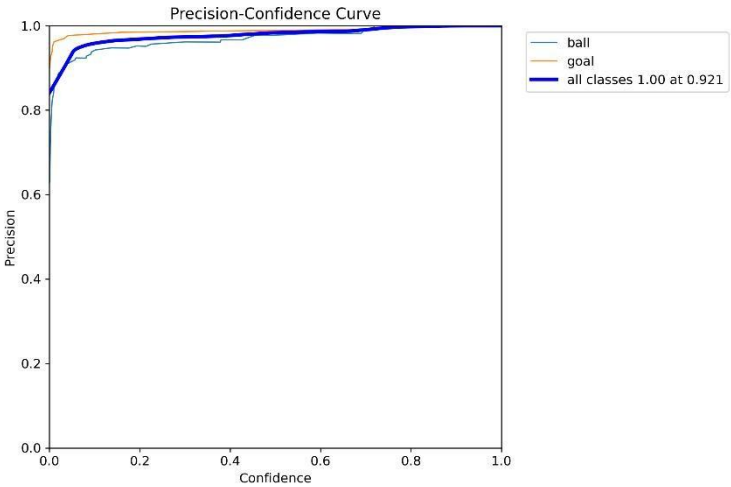


Figure 10. Precision curve

The curve shown in Figure 10 illustrates the relationship between precision and confidence threshold. Precision is a parameter that measures the accuracy of positive detections made by the YOLOv8 model [17], while confidence represents how certain the model is that an object exists [18]. The curve indicates that precision reaches a value of 1.00 at a confidence threshold of 0.921. This means that at this threshold, all detections are accurate, with no false positives. The curve also shows that precision remains consistently high at higher confidence thresholds, indicating that the model is highly reliable in avoiding incorrect detections. However, at lower confidence threshold, there are minor fluctuations, suggesting the presence of some false positives. This pattern highlights the trade-off between confidence and recall, where increasing

the confidence threshold improves precision but may reduce the number of correctly detected objects. For practical implementation, setting a confidence threshold of 0.883 ensures the highest detection accuracy, but a slightly lower threshold, such as 0.7 to 0.8, may be preferable to maintain a balance between precision and recall.

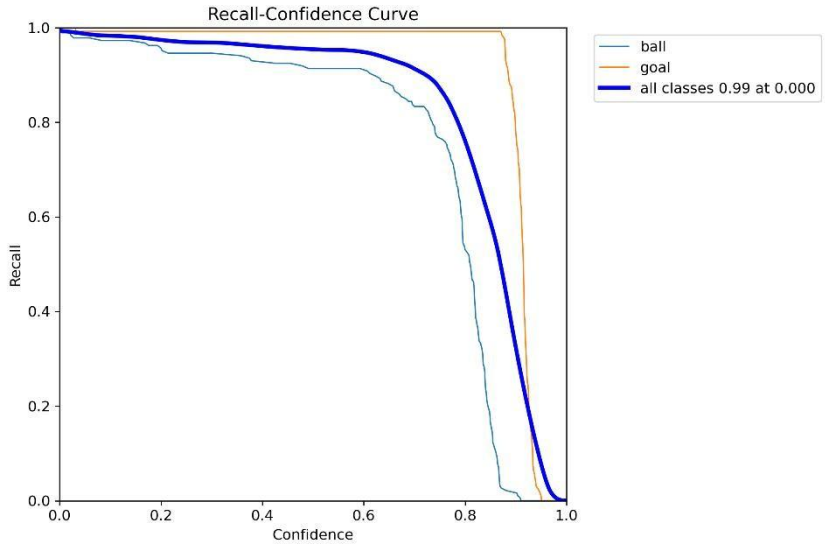


Figure 11. Recall curve

The curve shown in Figure 11 presents the recall value for each class concerning confidence thresholds. Recall represents the model's ability to detect objects that should be identified [19]. It is calculated as the number of correctly detected objects divided by the total number of objects present. At low confidence threshold, recall remains high for both classes, indicating that the model detects most objects but with varying certainty. However, as the confidence threshold increases, the recall for ball declines more rapidly compared to goalpost, which maintains a high recall until nearly 0.9 confidence. This suggests that goalpost is detected with higher certainty, while ball detections are more affected by the confidence threshold. The overall recall for all classes starts at 0.99 but decreases sharply after 0.8 confidence, highlighting a trade-off between detection completeness and certainty.

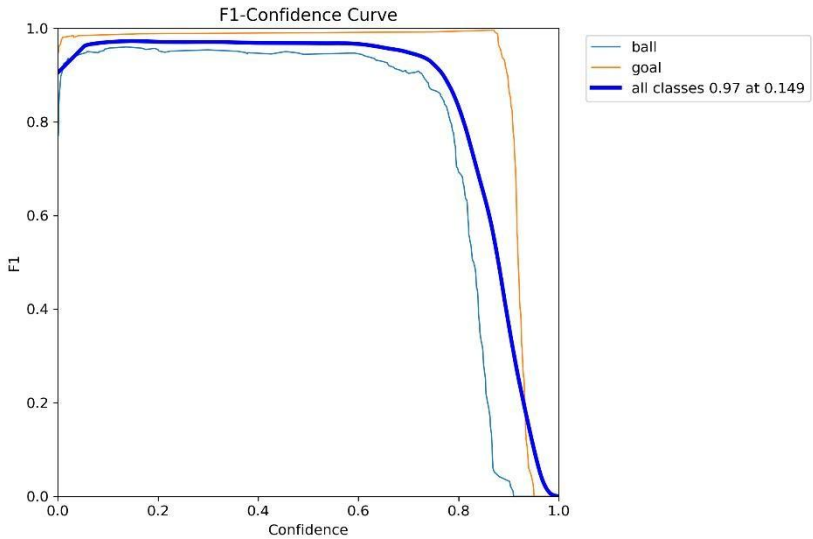


Figure 12. F1-Score curve

Figure 12 illustrates a curve graph representing the F1-Score values in relation to confidence thresholds. The F1-Score is a metric that combines precision and recall into a single value [20], providing a balanced measure of a model’s performance. A high F1-Score indicates that the

model has both strong precision (fewer false positives) and strong recall (fewer false negatives), making it a reliable indicator of classification performance [21]. From the graph, it is observed that the F1-Score reaches its highest value of 0.97 when the confidence threshold is at 0.149. However, it starts to decline sharply when confidence surpasses 0.8. This suggests that at lower confidence thresholds, the model maintains a good balance between precision and recall, meaning it can correctly detect a high number of objects. However, as the confidence threshold increases to 0.8, the model becomes more selective, discarding images that contain objects but are not distinctly clear. This results in a drop in recall, as some true positives are overlooked, affecting the overall F1-Score. A closer analysis of the class-specific curves (ball and goalpost) reveals that the goalpost class maintains a higher F1-Score across a broader confidence range compared to ball. This suggests that the model is more confident in detecting goalpost than ball, likely due to clearer object features or more consistent training data for goalpost. The ball class, on the other hand, experiences a steeper decline in F1-Score as confidence increases, indicating that it is more susceptible to misclassification at higher thresholds. This behavior may stem from lighting variations, occlusion, or similarities between the ball and background elements, which complicate detection.

CONCLUSION

Based on the conducted research, ball and goal detection using an omnidirectional camera on the KRSBI-Beroda robot can be effectively performed using the You Only Look Once (YOLO) method. After evaluating the model, the model's accuracy reaches 0.95. On the other hand, precision-confidence curve indicates that precision reaches 1.00 at a confidence threshold of 0.921, meaning that at this threshold, all detections made by the model are correct, with no false positives. Meanwhile, the recall-confidence curve shows that recall reaches 0.99 even at a low confidence threshold, demonstrating the model's ability to detect nearly all given objects. The F1-Score curve peaks at 0.97 when confidence is at 0.149 but begins to decline sharply after 0.8, indicating that the model performs optimally at moderate confidence thresholds before becoming overly selective.

ACKNOWLEDGMENT

I would like to express my heartfelt gratitude to everyone who supported the completion of this article. Although I am unable to mention each individual by name, their contributions, encouragement, and assistance are sincerely appreciated.

REFERENCES

BIOGRAPHIES OF AUTHORS